

Online Supplement: Extensions of LLM Inference Scheduling to Parallel Machines

1. Extensions to Parallel Machines

This section extends the single-machine results of the main paper to m identical parallel machines, each with KV-cache capacity M . We first extend the job squashing relaxation to a job splitting relaxation, which connects LLM inference scheduling in the parallel-machine setting to classical scheduling theory and provides the basis for lower-bound analysis and approximation algorithm design. We then extend the List Scheduling framework, which processes jobs according to a given priority order and assigns each job to a machine at its earliest feasible start time. This extension yields corresponding upper bounds on job start and completion times. Building on these structural results, the priority rules developed for both the single-agent and multi-agent settings extend naturally to parallel machines, largely preserving the same approximation guarantees, while all resulting algorithms admit $O(mn^3)$ -time implementations, where n denotes the number of jobs. Finally, we develop a machine-partitioning strategy that separates tall and low jobs across dedicated groups of machines and yields constant-factor approximation guarantees independent of β .

Table 1 summarizes the approximation results established in this paper.

1.1. Job Splitting Relaxation

We first generalize the job-squashing relaxation, which underlies our single-machine lower bounds, to the parallel-machine setting, in which the available capacity is scaled from M to the aggregate capacity mM . We then show that the corresponding structural bound and the lower bounds it yields carry over to parallel machines. These bounds serve as the foundation for extending the LS-based algorithms and their approximation guarantees to the parallel-machine setting.

Recall from the main paper (Definition 1) that the *job squashing relaxation* of an instance $\mathcal{I}$, denoted by $\mathcal{S}(\mathcal{I})$, is a classical single-machine scheduling problem on the same set of n jobs, where each job J_j has a *squashed processing time* $\tilde{p}_j = \frac{v_j}{M}$. For m identical parallel machines, we introduce the following extension.

Table 1 Summary of approximation results for the parallel-machine extensions.

Problem	Algorithm	Ratio	Extension
<i>Single-agent setting</i>			
$Pm \mid LLM \mid C_{\max}$	List Scheduling (any order)	$1 + \frac{2}{\alpha(\beta, \gamma)}$	Thm. 3
	LHF	$\min\{6, 1 + \frac{2}{\alpha(\beta, \gamma)}\}$	Thm. 4
$Pm \mid LLM \mid \sum C_j$	SVF	$1 + \frac{2}{\alpha(\beta, \gamma)}$	Thm. 5
	SVF (h-tie)	$1 + \frac{2}{\alpha(\beta, \gamma)}; 4^\ddagger$	Thm. 6
	SVF (h-tie) ($s_j = s$, all j)	$\frac{6}{6}$	Thm. 7
$Pm \mid LLM \mid \sum w_j C_j$	WSVF	$1 + \frac{2}{\alpha(\beta, \gamma)}$	Thm. 5
	Partitioned WSVF [†]	$\frac{10}{10}$	new
<i>Multi-agent setting</i>			
$Pm \mid LLM \mid \sum C_{0,j} : \{C_{\max}^h\}$	JSR-Mapping	$(1 + \frac{2}{\alpha(\beta, \gamma)}, \dots)$	Thm. 9
	Partitioned JSR-Mapping [†]	$(10, \dots, 10)$	new

$$\beta = M / \max_j (s_j + p_j), \quad \gamma = M / \min_j s_j, \quad \alpha(\beta, \gamma) = \max\{1 - 1/\beta + 1/\gamma, 2/\gamma\}.$$

[‡] The ratio 4 is asymptotic, attained when M is fixed and $n \rightarrow \infty$.

[†] Requires $m \geq 2$ machines; the ratio 10 is independent of β .

The WSVF guarantee reduces to the single-machine case $1 \mid LLM \mid \sum w_j C_j$ when $m = 1$.

All parallel-machine algorithms run in $O(mn^3)$ time.

LS = List Scheduling; SVF = Smallest-Volume-First; SVF (h-tie) = SVF with height-based tie-breaking; LHF = Largest-Height-First; WSVF = Weighted-Smallest-Volume-First; JSR-Mapping = Job Squashing Relaxation Mapping (Job Splitting Relaxation Mapping on parallel machines).

Except for Partitioned WSVF and Partitioned JSR-Mapping, the approximation ratios of all other algorithms are direct extensions of the corresponding guarantees established in the main paper.

DEFINITION E.1 (JOB SPLITTING RELAXATION). The *job splitting relaxation* of $\mathcal{I}$, denoted by $\mathcal{S}_m(\mathcal{I})$, is a classical single-machine scheduling problem on the same set of n jobs, where each job J_j has a *split processing time* $\tilde{p}_j = \frac{v_j}{mM}$.

Thus, $\mathcal{S}_m(\mathcal{I})$ differs from $\mathcal{S}(\mathcal{I})$ only in that the available KV-cache capacity is scaled from M to the aggregate capacity mM . The following theorem extends the job-squashing bound (Theorem 2 in the main paper) to m parallel machines.

THEOREM E.1 (Job Splitting Bound). Consider any instance $\mathcal{I}$ and its job-splitting relaxation $\mathcal{S}_m(\mathcal{I})$, as defined in Definition E.1. Consider any feasible schedule of $\mathcal{I}$ on m identical parallel machines, and let $\sigma = (\sigma_k)_{k=1}^n$ denote the job indices ordered by non-decreasing completion times, breaking ties arbitrarily. Let C'_{σ_k} denote the completion time of job J_{σ_k} when the jobs of $\mathcal{S}_m(\mathcal{I})$ are processed consecutively, without idle time, in the same order σ . Then, for each $k \in \{1, \dots, n\}$, the completion time C_{σ_k} of J_{σ_k} in the original schedule is bounded from

below by its completion time C'_{σ_k} in the relaxed schedule, i.e.,

$$C_{\sigma_k} \geq C'_{\sigma_k} = \frac{1}{mM} \sum_{i=1}^k v_{\sigma_i}.$$

The proof mirrors that of the job-squashing bound (Theorem 2 in the main paper), with the single-machine capacity M replaced by the aggregate capacity mM . Consider the first k jobs to complete in the original parallel-machine schedule. By time C_{σ_k} , these jobs must have collectively consumed their full volumes, whose total is $\sum_{i=1}^k v_{\sigma_i}$. Since the aggregate KV-cache capacity of the m machines is mM , the elapsed time required to accommodate this total volume is at least $\frac{1}{mM} \sum_{i=1}^k v_{\sigma_i}$. This is exactly the completion time C'_{σ_k} of J_{σ_k} in the job-splitting relaxation when its jobs are processed consecutively in order σ without idle time.

The job-splitting bound immediately yields the following parallel-machine counterparts of the lower bounds established in Corollary 1 of the main paper.

COROLLARY E.1. *For the total weighted completion time objective, sort jobs in non-decreasing order of v_j/w_j , breaking ties arbitrarily, and let $\sigma = (\sigma_k)_{k=1}^n$ denote the resulting permutation. Then*

$$\sum_{j=1}^n w_j C_j^*(I) \geq \frac{1}{mM} \sum_{k=1}^n w_{\sigma_k} \sum_{i=1}^k v_{\sigma_i}.$$

For the makespan objective,

$$C_{\max}^*(I) \geq \frac{1}{mM} \sum_{j=1}^n v_j.$$

The job-splitting relaxation also extends naturally to the multi-agent setting: for each job $J_{h,j}$ of Agent $\mathcal{A}_h$, we assign the split processing time $\tilde{p}_{h,j} = \frac{v_{h,j}}{mM}$. Thus, the multi-agent job-splitting relaxation is obtained from the single-machine multi-agent squashing relaxation by replacing the available capacity M with the aggregate capacity mM . The following corollary extends Corollary 2 to m parallel machines.

COROLLARY E.2 (Multi-Agent Job Splitting Bound). *For an instance I of the multi-agent LLM scheduling problem, the minimal total completion time of Agent $\mathcal{A}_0$ in the job-splitting relaxation $\mathcal{S}_m(I)$ is no larger than that of the original parallel-machine instance I , i.e., $\sum_j C_{0,j}^*(\mathcal{S}_m(I)) \leq \sum_j C_{0,j}^*(I)$.*

1.2. Extension of List Scheduling

The List Scheduling (LS) framework extends to m identical parallel machines. Given a priority sequence σ , when a job is considered, LS evaluates its earliest feasible start time on each of the m machines independently and assigns the job to the machine attaining the smallest such start time. Since this computation is performed separately on each machine, the running-time

analysis of Proposition 1 carries over directly: LS admits an $O(mn^3)$ -time implementation on m parallel machines for any priority sequence σ .

Crucially, Lemma 1 extends to the parallel-machine setting, providing the structural start-time bound underlying our subsequent approximation analyses.

LEMMA E.1. *Consider any priority sequence $\sigma = (\sigma_k)_{k=1}^n$ of the job indices. Let $\{S_j\}_{j=1}^n$ be the start times returned by LS on m identical parallel machines, each with KV-cache capacity M . Then, for each $k \in \{1, \dots, n\}$,*

$$S_{\sigma_k} < \frac{2 \sum_{i=1}^{k-1} v_{\sigma_i}}{m\alpha(\beta, \gamma)M}.$$

The proof applies the single-machine bound of Lemma 1 to the potential start time of the current job on each machine and averages over the m machines; this replaces the factor $1/M$ by $1/(mM)$, reflecting the aggregate KV-cache capacity of the parallel machines. The key structural property is thus preserved: the start time of each job is controlled by the cumulative volume of previously considered jobs, scaled by mM , so that the approximation guarantees built on Lemma 1 carry over to the parallel-machine setting.

The universal per-job completion-time bound for List Scheduling established in Lemma 2 also carries over to the parallel-machine setting. To state this extension, recall the block decomposition of the main paper: given a priority sequence σ , a job is *tall* if $h_j > M/2$ and *low* otherwise, and σ is partitioned into maximal contiguous blocks of alternating type; let b_j denote the block containing J_{σ_j} and $p_{\max}^{(k)}$ the maximum processing time in block k . The completion-time bound in Lemma 2 extends to the parallel-machine setting by invoking the start-time bound in Lemma E.1, yielding the following result.

LEMMA E.2. *Let $\sigma = (\sigma_1, \dots, \sigma_n)$ be any priority sequence of the job indices, and let $\{S_{\sigma_j}\}_{j=1}^n$ and $C_{\sigma_j} = S_{\sigma_j} + p_{\sigma_j}$ denote the start and completion times returned by LS on m identical parallel machines. Partition σ conceptually into alternating tall/low blocks as defined above, and adopt the block notation b_j and $p_{\max}^{(k)}$. Then, for each job J_{σ_j} ,*

$$C_{\sigma_j} \leq \frac{4}{mM} \sum_{k=1}^j v_{\sigma_k} + p_{\sigma_j} + \sum_{k=1}^{b_j-1} p_{\max}^{(k)}.$$

The proof is analogous to that of Lemma 2, with the single-machine start-time bound replaced by Lemma E.1; consequently, the cumulative-volume term is scaled by the aggregate capacity mM . The same completion-time structure is thus preserved, with the contribution of previously completed blocks remaining unchanged.

1.3. Extension of Approximation Algorithms

Building on these two extensions, the algorithms for the single-machine setting carry over to m identical parallel machines with the same analytical structure and analogous approximation guarantees. For the makespan objective, List Scheduling under any priority order achieves an approximation ratio of $1 + 2/\alpha(\beta, \gamma)$, where $\beta = M/\max\{h_j : J_j \in \mathcal{J}\}$ and $\gamma = M/\min_j s_j$ characterize the KV-cache capacity relative to the maximum job memory footprint and the minimum job prompt size, respectively, and $\alpha(\beta, \gamma) := \max\{1 - 1/\beta + 1/\gamma, 2/\gamma\}$. The Largest-Height-First (LHF) rule refines this to $\min\{6, 1 + 2/\alpha(\beta, \gamma)\}$. For the total completion time objective, the Smallest-Volume-First (SVF) rule likewise achieves a ratio of $1 + 2/\alpha(\beta, \gamma)$; refining SVF with height-based tie-breaking further yields an asymptotic ratio of 4, and a ratio of 5 when all jobs share the same prompt size. Moreover, for the total weighted completion time objective, the Weighted Smallest-Volume-First (WSVF) rule achieves a ratio of $1 + 2/\alpha(\beta, \gamma)$, which reduces to the single-machine guarantee when $m = 1$.

The same extension applies to the multi-agent setting. Specifically, the Job Squashing Relaxation Mapping (JSR-Mapping) algorithm extends to a Job-Splitting-Relaxation Mapping algorithm by replacing the job-squashing relaxation with the job-splitting relaxation. The algorithm first exploits the optimal solution structure of the job-splitting relaxation (the parallel analogue of Lemma 3) to construct the priority order, and then applies List Scheduling to this order on the m parallel machines. Using the parallel-machine bounds above yields the vector-valued approximation guarantee $\left(1 + \frac{2}{\alpha(\beta, \gamma)}, \dots, 1 + \frac{2}{\alpha(\beta, \gamma)}\right)$.

Finally, for $m \geq 2$, we obtain constant-factor guarantees independent of β through a machine-partitioning strategy. The key idea is to separate tall and low jobs and dedicate a distinct group of machines to each job type. Specifically, partition the m machines into two groups $\mathcal{G}_1$ and $\mathcal{G}_2$ with $|\mathcal{G}_1| = \kappa$ and $|\mathcal{G}_2| = m - \kappa$, where $\kappa = \lceil m/2 \rceil$, and split the jobs into

$$\mathcal{J}_{\text{low}} = \left\{ J_j \mid \frac{s_j + p_j}{M} \leq \frac{\kappa}{m} \right\}, \quad \mathcal{J}_{\text{tall}} = \left\{ J_j \mid \frac{s_j + p_j}{M} > \frac{\kappa}{m} \right\}.$$

We then schedule the low jobs on $\mathcal{G}_1$ and the tall jobs on $\mathcal{G}_2$, using the WSVF priority order and List Scheduling within each group. This partitioning strategy preserves the volume-based analysis for low jobs and the processing-time bound for tall jobs, and yields a constant approximation ratio of 10 for the total weighted completion time objective, independent of β .

The same partitioning strategy also extends to the multi-agent setting. Specifically, we apply the Job-Splitting-Relaxation Mapping framework with the same partition of machines and jobs. Within each machine group, List Scheduling is then used to construct the corresponding parallel-machine schedules for the low and tall jobs. The same analysis as in the single-agent case yields the vector-valued approximation guarantee $(10, \dots, 10)$, which is independent of β .